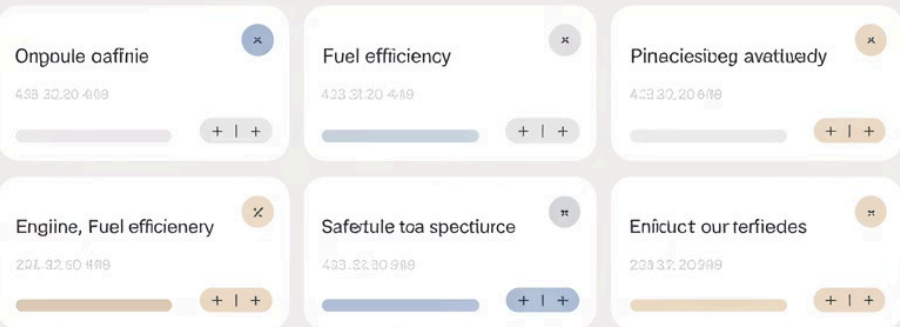




Let's find your perfect ride



多輪引導式漏斗框架實作計畫

中小型房車銷售智能助理

本簡報詳細說明中小型房車銷售智能助理的技術架構與實作計畫，透過多輪對話引導客戶完成購車決策。

簡報大綱

01

核心模組概覽

對話狀態管理、對話控制、槽位填充、推薦引擎等八大核心模組

03

潛在挑戰

LLM路由錯誤、資料時效性等五大挑戰

02

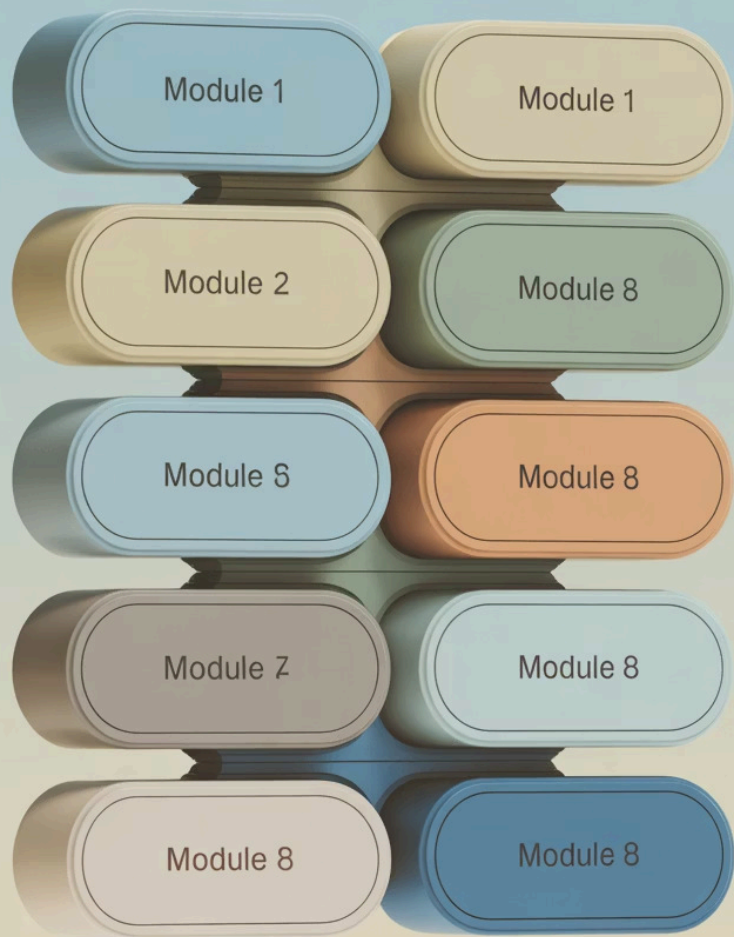
技術棧

Python、LangChain、LangGraph等核心技術與工具

04

實作階段

從資料準備到測試優化的八個實作階段



核心模組概覽

本系統由八個核心模組組成，共同協作以實現智能化的銷售漏斗流程：

- 對話狀態管理模組 (DST)
- 對話控制模組 (DM)
- 槽位填充引擎
- 推薦引擎
- 提示鏈模組 ("Think, Then Act")
- 車款知識庫
- 長期記憶與個人化模組
- 錯誤處理與對話修復模組

對話狀態管理模組 (DST)

目的

維護最新的對話上下文、銷售漏斗階段、已收集的槽位值，以及推薦記錄。

主要欄位

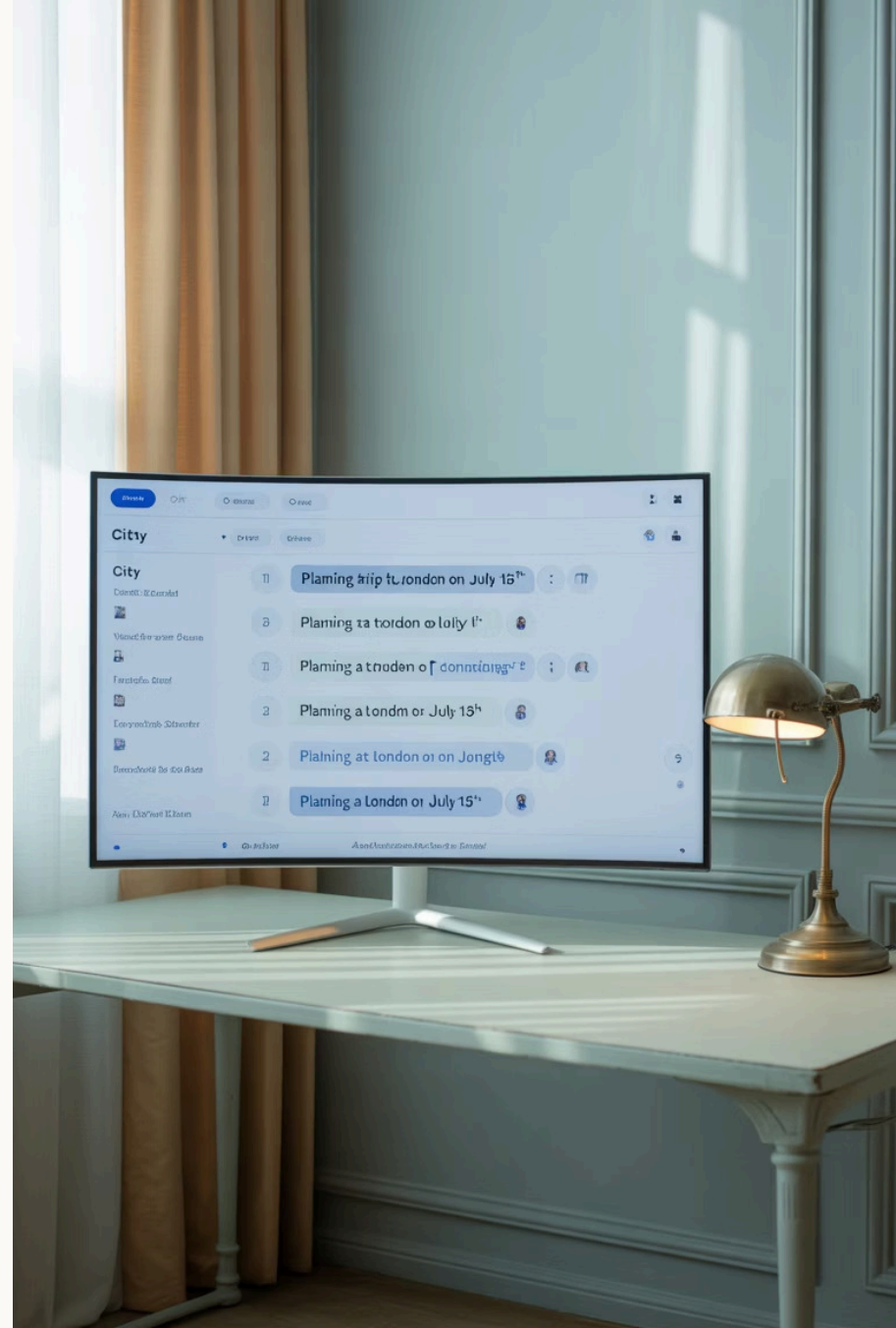
- **chat_history**：逐輪對話訊息
- **filled_slots**：槽位名稱 → 值的字典
(例如 budget、seat_count、usage)
- **recommendations**：已建議的產品 ID/名稱列表

技術實現

- Python TypedDict
- LangGraph Stateful Graph
- LangChain Memory

潛在問題

- 狀態遺失
- 槽位結構不完整
- 槽位鍵與定義不一致



對話控制模組 (DM)

目的

根據當前對話狀態決定下一步系統行動，是整個對話流程的「大腦」。

主要功能

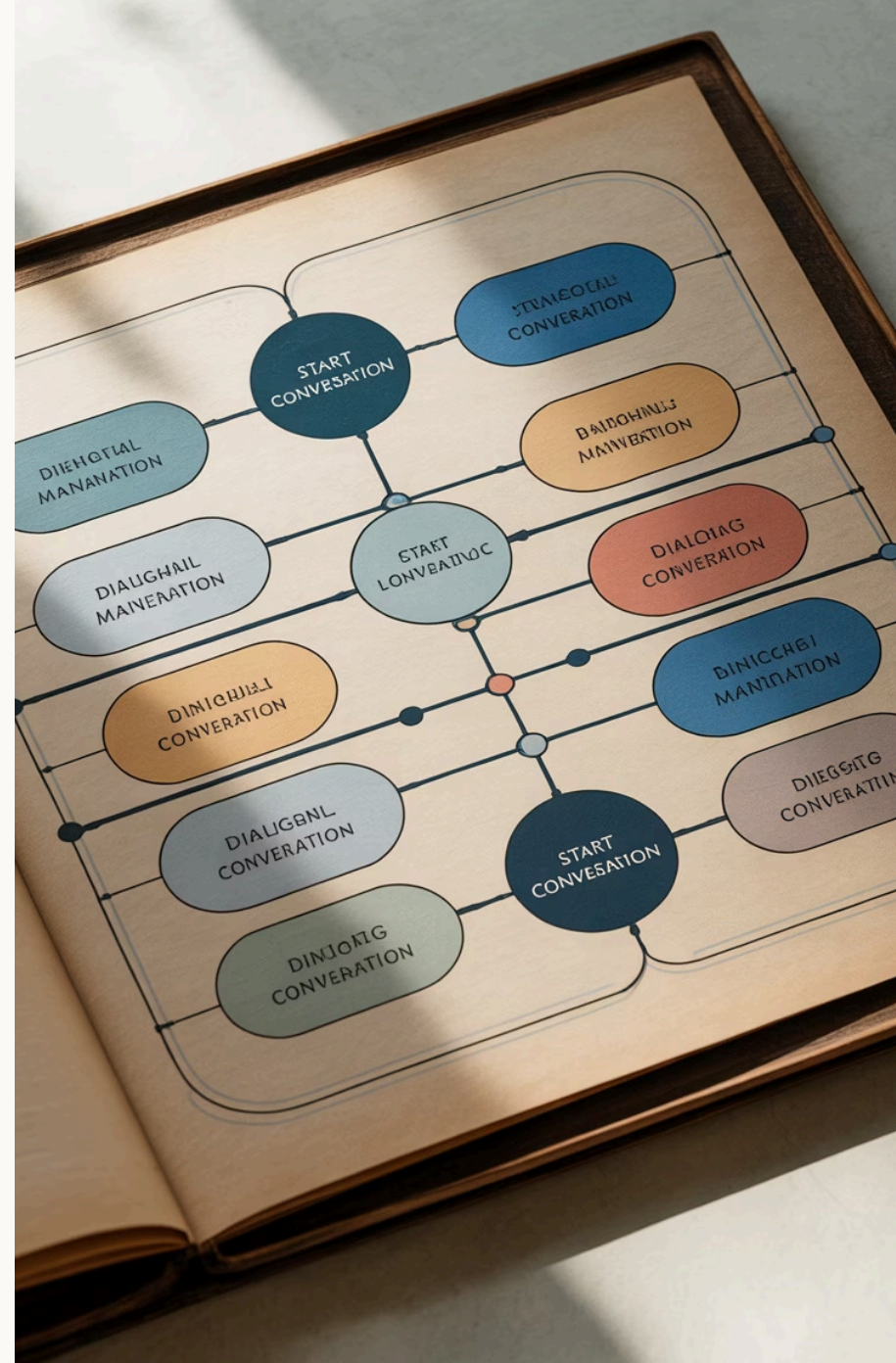
- 對話流程路由 (Think 步驟)
- 執行商業漏斗邏輯

技術實現

- LangGraph Conditional Edges
- LangChain PromptTemplate
- 支援結構化輸出的 LLM

潛在問題

- 用戶意圖判斷錯誤
- 對話偏離漏斗路徑



槽位填充引擎

目的

蒐集購車關鍵需求（例如預算、座位數、燃料類型、品牌偏好）。

主要特色

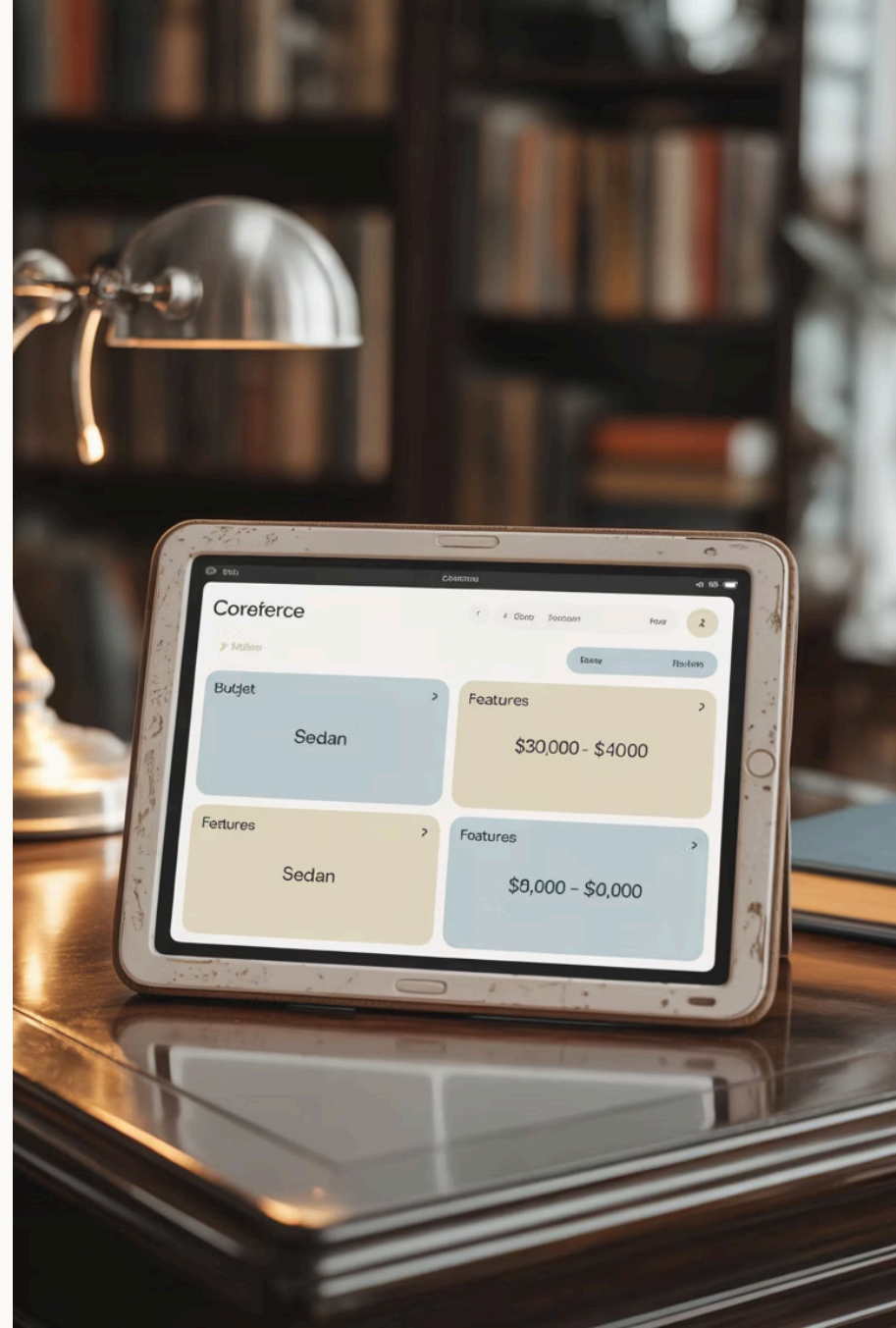
- 必填/選填標記
- 多值槽位
- 澄清問題模板

技術實現

- 基於 LLM 的實體抽取
- 提示鏈
- LangChain 工具調用

潛在問題

- 用戶輸入模糊
- 同義詞對應
- 缺乏必要澄清



槽位填充示例



用戶輸入

「我想找一台預算在80萬以內，適合家庭使用的車子，最好是省油的。」



槽位抽取

budget: "80萬以內"
usage: "家庭使用"
feature: "省油"



澄清問題

「您家庭有幾位成員需要乘坐？偏好五人座還是七人座的車款？」

系統會持續引導用戶填充必要的槽位，直到收集足夠資訊進行推薦。

推薦引擎

目的

篩選車款資料庫並生成有說服力的推薦。

實作方法

1. **基礎版**：程式過濾
2. **進階版**：基於 RAG 的語義檢索 + LLM 綜合生成

技術實現

- Pandas
- FAISS/Chroma
- LangChain RetrievalQA

潛在問題

- 相關性排序不佳
- 生成虛構規格

Desicortc

Home

Saved

Cars

Acquire

Eono

Accou

Enter make
model, or
feature

Enter make, model, or feature

Demursis Vivem sathisee sint aperts

Q. Butfloor oudnenddanc

Budget

Fuel Efficiency

Mileage

● Mileage

Efficiency

Efficiency

Efficiency

● Hybrid

Total Price

View Cars

View Details

Matching Cars



Matching Cars

2018-2019

15 - 20,000-90

Matching Cars

2018-2019

15 - 20,000-90

View Details

2018-2019

2018-2019

推薦引擎進階版流程

槽位值轉換為查詢

將用戶需求（預算、座位數等）轉換為結構化查詢

過濾與排序

根據硬性條件（如預算）過濾結果並排序

向量檢索

使用 FAISS/Chroma 進行語義相似度檢索

LLM 綜合生成

使用 LLM 生成有說服力的推薦文案，強調符合用戶需求的特點

進階版推薦引擎結合了傳統過濾與 LLM 生成能力，提供更個性化且有說服力的車款推薦。

提示鏈模組 ("Think, Then Act")

目的

將策略決策 (Think) 與面向用戶的回應生成 (Act) 拆分，降低 LLM 偏離風險。

技術實現

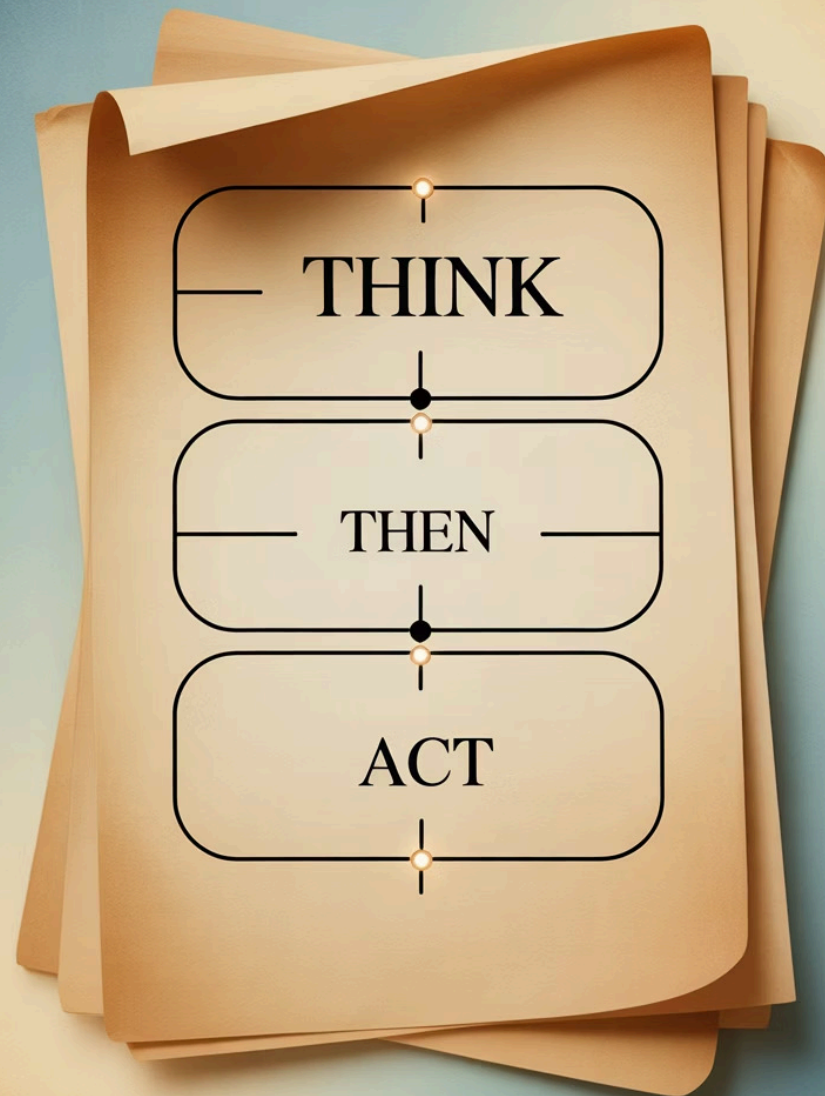
- LangChain ChatPromptTemplate
- 結構化輸出 (Pydantic 模型)

工作流程

1. **Think**：分析對話狀態，決定下一步行動
2. **Act**：根據決策生成自然、有幫助的回應

潛在問題

- 模板變數錯誤
- 提示過於寬鬆導致偏離目標



提示鏈模組示例



Think 階段

輸入：對話歷史、已填槽位

輸出：`{"action": "ask_for_slot", "slot": "fuel_type", "reason": "用戶已提供預算和座位需求，但未說明燃料偏好"}`



Act 階段

輸入：Think 輸出、對話歷史

輸出：「感謝您提供的資訊。為了幫您找到最合適的車款，請問您對燃油車、油電混合車或電動車有特別偏好嗎？」

這種拆分方式確保系統能夠保持對話流程的控制，同時生成自然、友善的回應。



車款知識庫

目的

儲存中小型房車的詳細規格，供搜尋與檢索。

資料結構

CSV，欄位包含品牌、型號、座位數、燃料類型、價格區間、功能特色等。

技術實現

- Pandas 載入 CSV
- 向量資料庫做 RAG

潛在問題

- 資料更新不及時
- 不同品牌欄位結構不一致

車款知識庫示例結構

品牌	型號	價格區間	座位數	燃料類型	特色
Toyota	Corolla Cross	80-95萬	5	油電混合	省油、空間大、安全配備完善
Honda	HR-V	85-110萬	5	汽油	操控性佳、Magic Seat彈性空間
Mazda	CX-30	90-120萬	5	汽油	精緻內裝、駕駛樂趣、安全科技

知識庫需定期更新以反映最新車款資訊、價格變動及促銷活動。向量化後的知識庫能支援更靈活的語義檢索。



長期記憶與個人化模組

目的

跨會話保存用戶資料，提供個性化體驗。

方法

以 `user_id` 為鍵，將 `DialogueState` 存於 Redis 或 PostgreSQL。

技術實現

- SQLAlchemy
- Redis
- LangGraph Checkpointing

潛在問題

- 隱私與合規問題
- 用戶身份辨識可靠性

長期記憶應用場景

回訪識別

「歡迎回來！上次您對Toyota Corolla Cross表示了興趣，我們有新的優惠方案可以分享。」

偏好記憶

「根據您過去的偏好，我們推薦這些省油且安全性高的車款...」

購買歷程續接

「上次我們討論到您需要一台適合家庭的五人座車款，您考慮得如何？需要更多資訊嗎？」

長期記憶模組讓系統能夠提供更連貫、個性化的服務體驗，增加用戶滿意度和轉換率。

錯誤處理與對話修復模組

目的

處理離題問題、誤解、或用戶情緒負面情況。

策略

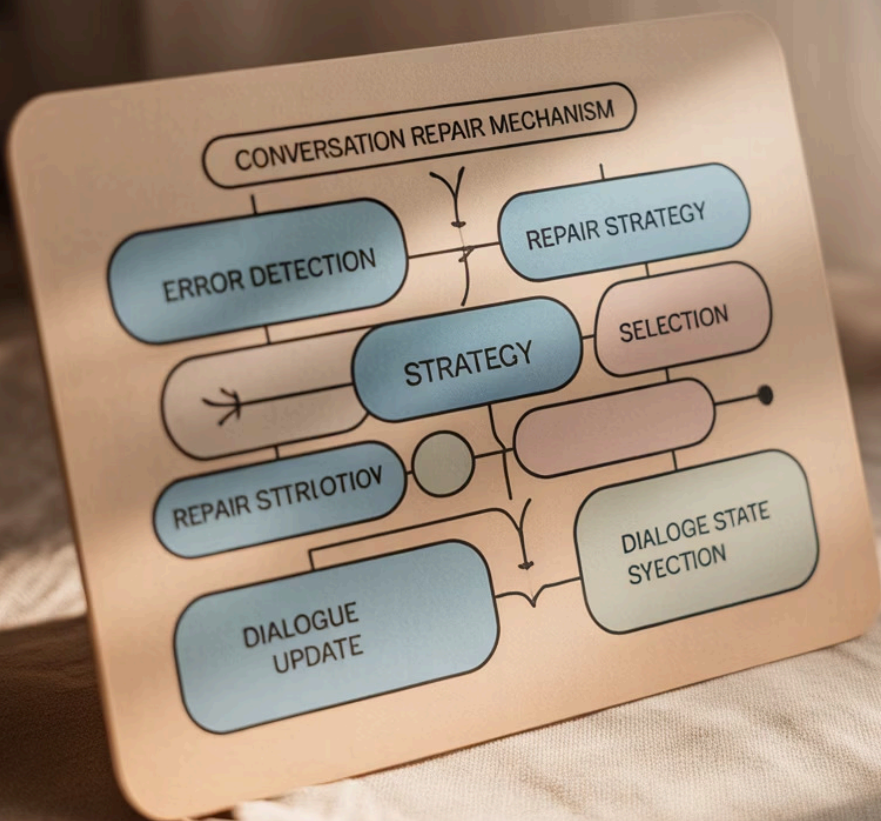
- 重述問題
- 提供多選選項
- 支援用戶中斷與切換需求
- 必要時升級至人工客服

技術實現

- 情感分析
- LangGraph 後備路由

潛在問題

- 過度打斷用戶
- 修復策略顯得機械



錯誤處理示例



用戶輸入

「這些車款都太貴了，你們有沒有更便宜的選擇？而且我覺得你沒有理解我的需求。」



問題檢測

系統檢測到：1) 價格不滿意 2) 可能的溝通誤解
3) 輕微負面情緒



修復回應

「非常抱歉沒有完全理解您的需求。讓我們重新確認一下：您希望找到更經濟實惠的選擇。請問您的預算範圍是多少？我會根據這個調整推薦。」

有效的錯誤處理能挽回可能流失的對話，提高整體轉換率。

技術棧概覽



核心語言

Python



資料處理

Pandas



結構化輸出驗證

Pydantic



框架

LangChain、LangGraph



向量資料庫

FAISS / Chroma



LLM 供應商

OpenAI / Anthropic



資料持久化

Redis / PostgreSQL

LangChain 與 LangGraph

LangChain

用於構建 LLM 應用的框架，提供：

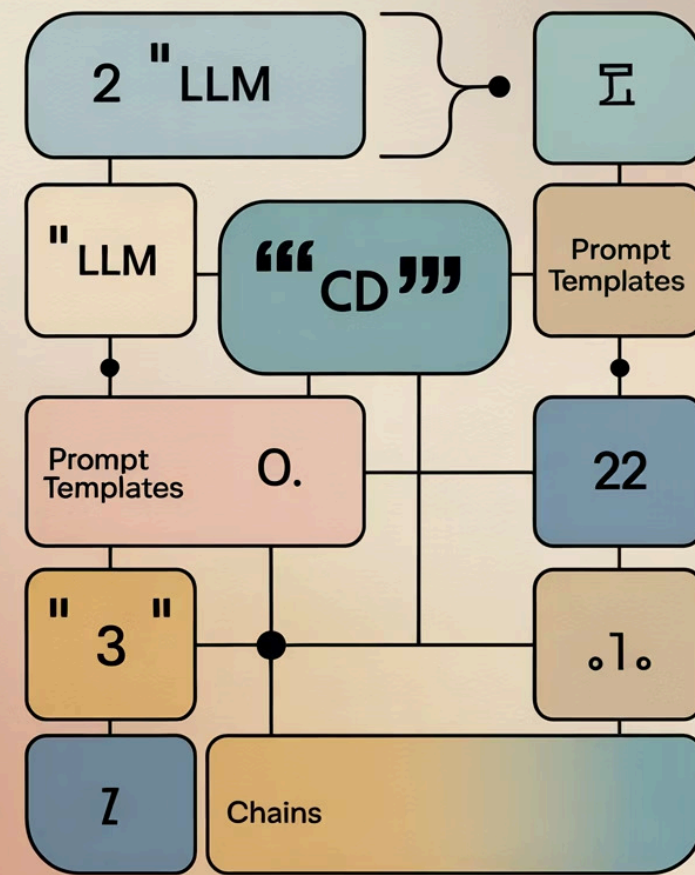
- 提示模板管理
- LLM 調用抽象層
- 記憶組件
- 工具與代理

LangGraph

基於 LangChain 的狀態化工作流引擎：

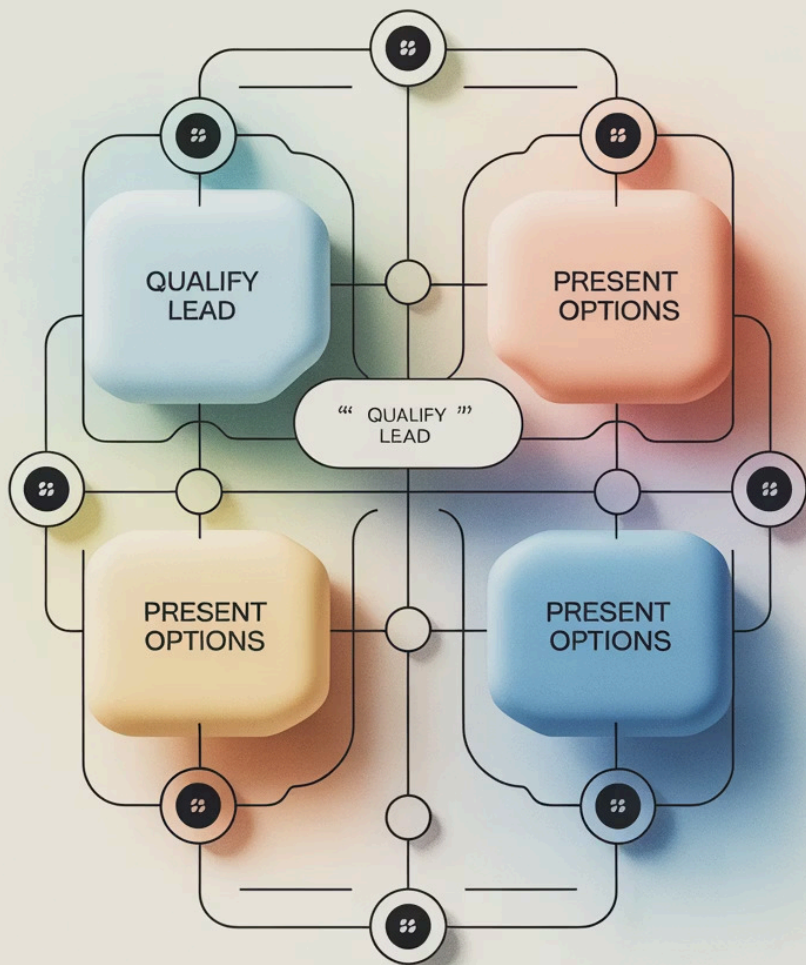
- 有向圖表示複雜流程
- 條件邊緣與路由
- 狀態管理與持久化
- 循環與反饋機制

LanGChain LangGraph



LANGGRAPH

CAR SALES



LangGraph 對話流程示例

LangGraph 允許我們將對話流程表示為有向圖，其中：

- **節點**：代表處理步驟（如 Think、Act、槽位填充、推薦）
- **邊緣**：定義節點間的轉換條件
- **狀態**：在整個流程中保持一致的對話狀態

這種結構使複雜的多輪對話邏輯變得可管理且可擴展。



潛在挑戰

實作過程中需要克服的五大關鍵挑戰

挑戰 2：資料時效性

問題描述

車款知識庫需定期更新，否則可能推薦已停產車型或提供過時價格資訊。

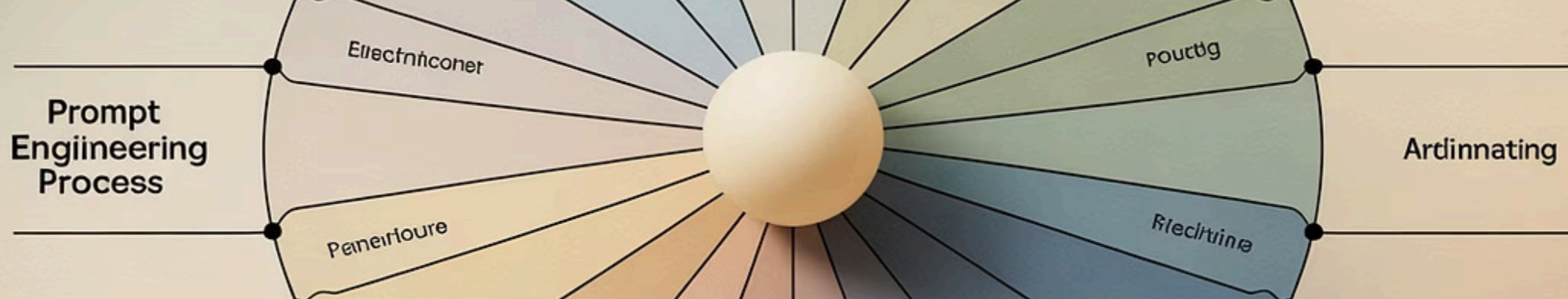
影響

用戶信任度下降，可能導致銷售機會流失。

緩解策略

- 建立自動化資料更新管道
- 實作資料版本控制
- 在回應中標記資料更新時間
- 設置資料過期警告機制





挑戰 3：提示穩定性

問題描述

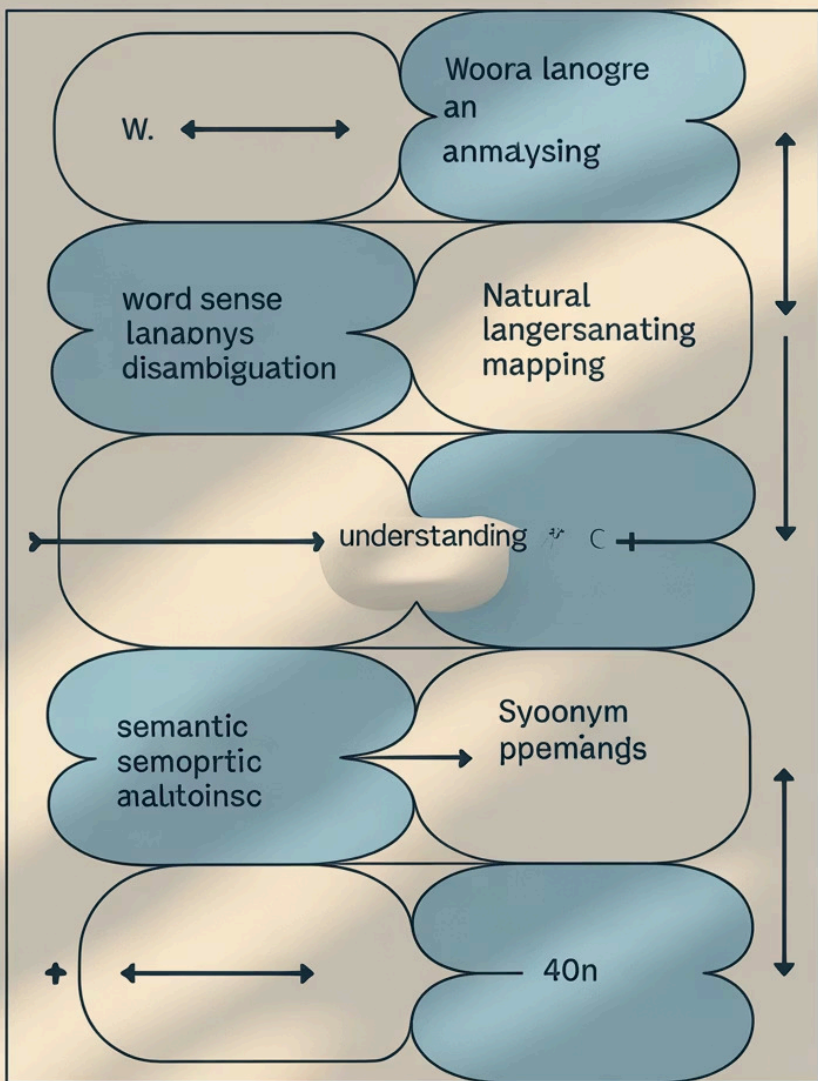
在銷售語氣與自然對話之間找到平衡，避免過度推銷或過於被動。

影響

用戶感知系統為純銷售工具而非助手，降低參與度。

緩解策略

- 精心設計提示模板，平衡銷售與協助
- A/B 測試不同語氣的效果
- 根據對話階段調整語氣
- 收集用戶反饋持續優化



挑戰 4：同義詞與模糊解析

問題描述

確保槽位填充能理解不同說法，如「經濟實惠」、「省油」、「便宜」等同義表達。

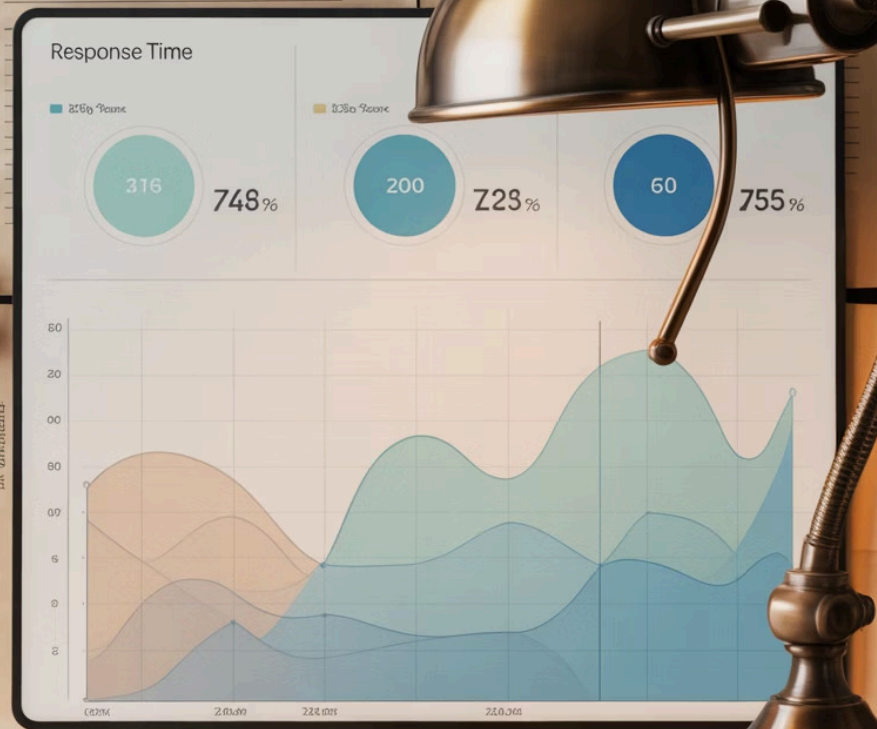
影響

槽位填充不完整或錯誤，導致推薦不準確。

緩解策略

- 建立同義詞詞典
- 使用更強大的 LLM 進行實體抽取
- 實作模糊匹配邏輯
- 主動確認模糊輸入

Performance Optimization



挑戰 5：效能

問題描述

RAG 與 LLM 請求可能導致延遲，影響實時對話體驗。

影響

用戶等待時間過長，可能放棄對話。

緩解策略

- 實作回應快取
- 使用非同步處理
- 優化向量檢索效率
- 實作打字動畫或思考指示器



實作階段

從資料準備到測試優化的八個關鍵階段

階段 1：資料準備

定義槽位結構

- 確定必填與選填槽位
- 設計槽位類型與驗證規則
- 建立槽位間的依賴關係

建立車款知識庫

- 收集中小型房車資料
- 標準化欄位格式
- 確保資料完整性與準確性

這個階段的品質直接影響後續所有功能的效果，需要投入足夠時間確保資料結構合理且完整。

槽位結構示例

槽位名稱	類型	必填	描述
budget	範圍	是	預算範圍，例如「50-80萬」
seat_count	整數	是	座位數需求，例如「5人座」
fuel_type	列舉	否	燃料類型偏好：汽油、柴油、油電混合、電動
brand_preference	多選	否	品牌偏好，例如「Toyota、Honda」
usage	文本	是	主要用途，例如「家庭使用」、「通勤」

完善的槽位結構是引導式對話的基礎，需要根據實際銷售經驗不斷優化。



階段 2：核心框架建立

實作 DialogueState 結構

使用 Python TypedDict 或 Pydantic 模型定義對話狀態結構，包含：

- chat_history
- filled_slots
- current_stage
- recommendations

初始化 LangGraph 狀態化流程

建立基本的對話圖結構：

- 定義節點（Think、Act等）
- 設置初始邊緣連接
- 實作狀態管理邏輯

DialogueState 結構示例

```
from typing import TypedDict, List, Dict, Optional

class DialogueState(TypedDict):
    # 對話歷史
    chat_history: List[Dict[str, str]] # 格式: [{"role": "user", "content": "..."}, ...]

    # 已填充槽位
    filled_slots: Dict[str, any] # 格式: {"budget": "50-80萬", "seat_count": 5, ...}

    # 當前階段
    current_stage: str # 例如: "greeting", "slot_filling", "recommendation", "closing"

    # 推薦記錄
    recommendations: List[str] # 已推薦的車款ID列表

    # 用戶滿意度追蹤
    satisfaction: Optional[int] # 1-5 評分，初始為 None
```

這種結構化的狀態定義確保了對話系統的各個組件能夠一致地訪問和更新對話信息。

階段 3：路由與提示鏈

建立 `route_action` 函數

先實作程式版本，後續升級為 LLM 版本

設計 **Act** 提示模板

根據 **Think** 輸出生成自然對話回應

設計 **Think** 提示模板

包含對話狀態分析和決策邏輯

整合提示鏈到 **LangGraph**

將提示鏈作為節點加入對話圖

提示鏈的設計是系統智能程度的關鍵，需要反覆測試和優化。

Think 提示模板示例

📘 你是一個專業的汽車銷售助理系統的決策引擎。你的任務是分析當前對話狀態，並決定下一步最合適的行動。

當前對話狀態：

- 已填充槽位：{filled_slots}
- 當前階段：{current_stage}
- 對話歷史（最近3輪）：{recent_history}

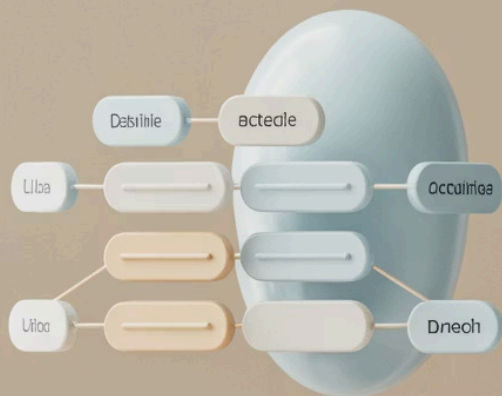
可選行動：

1. ask_for_slot：詢問特定槽位值
2. recommend：根據已有槽位提供推薦
3. clarify：澄清用戶意圖或輸入
4. provide_info：提供車款相關資訊
5. human_handoff：轉接人工客服

請分析情況並以JSON格式輸出你的決策：

```
{"action": "行動名稱", "parameters": {}, "reasoning": "決策理由"}
```

這種結構化提示確保 LLM 能夠做出符合銷售漏斗邏輯的決策，同時提供決策理由以便於調試和改進。



Enterateyodlanoir

Entity extraction

slot-filling
process

Llnthion

階段 4：槽位填充

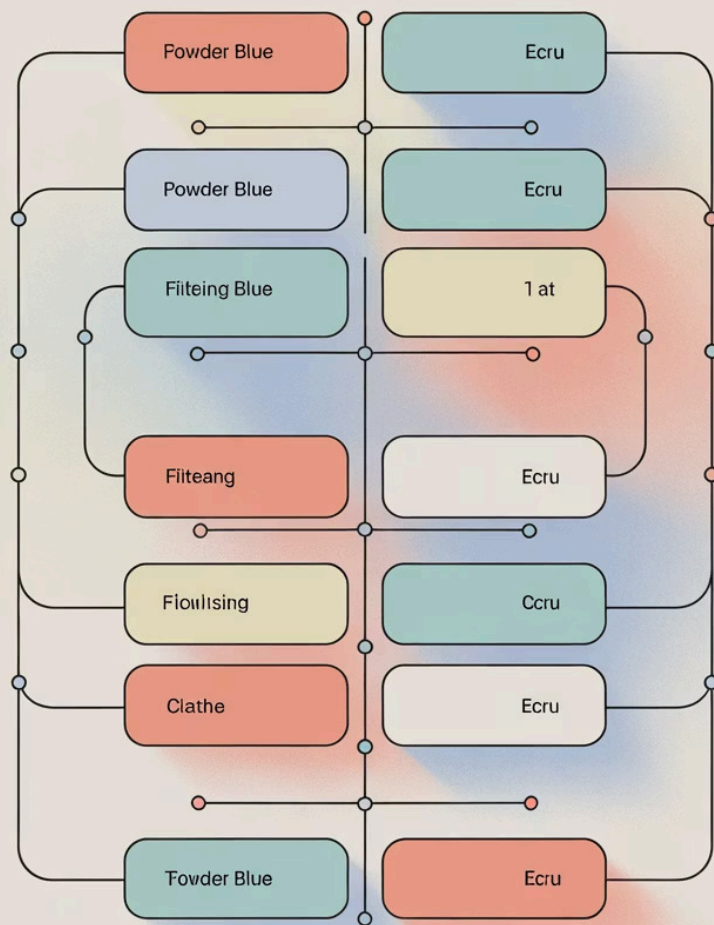
實作槽位抽取邏輯

- 設計槽位抽取提示
- 實作結構化輸出解析
- 建立槽位值驗證機制

建立澄清問題生成

- 設計澄清問題模板
- 實作模糊輸入檢測
- 建立多選選項生成邏輯

Recommendation Engine Pipeline Pipeline



階段 5：推薦邏輯

實作基礎篩選

- 根據硬性條件過濾車款
- 實作相關性排序
- 設計推薦回應模板

整合 RAG 管線

- 向量化車款描述
- 實作語義檢索
- 設計 LLM 綜合生成提示

階段 6：錯誤處理

增加離題檢測

實作檢測用戶問題是否偏離銷售主題的邏輯，並設計適當的回應策略。

實作對話修復

設計對話修復提示和流程，包括重述、澄清和提供選項等策略。

建立升級機制

實作檢測何時需要升級至人工客服的邏輯，以及無縫轉接的流程。

錯誤處理是提高系統魯棒性的關鍵，能夠顯著提升用戶體驗和轉換率。

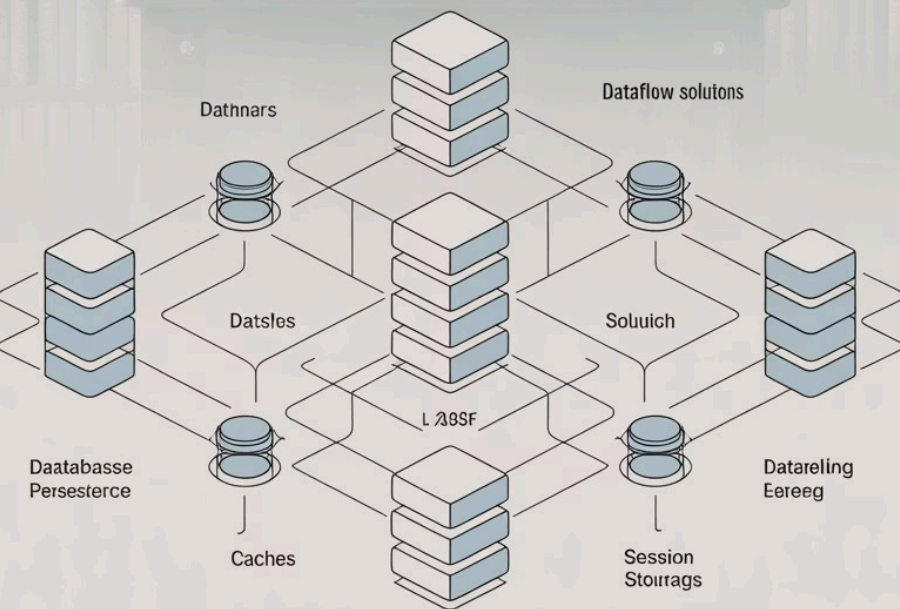
階段 7：持久化層

實作跨會話狀態保存

- 設計資料庫結構
- 實作序列化與反序列化邏輯
- 建立用戶識別機制

實作狀態讀取

- 實作會話初始化邏輯
- 設計狀態合併策略
- 建立資料過期機制



階段 8：測試與優化

模擬多種用戶路徑

設計測試腳本覆蓋各種可能的對話路徑，包括正常流程和邊緣情況。

調整提示模板

根據測試結果優化各個提示模板，提高系統回應的準確性和自然度。

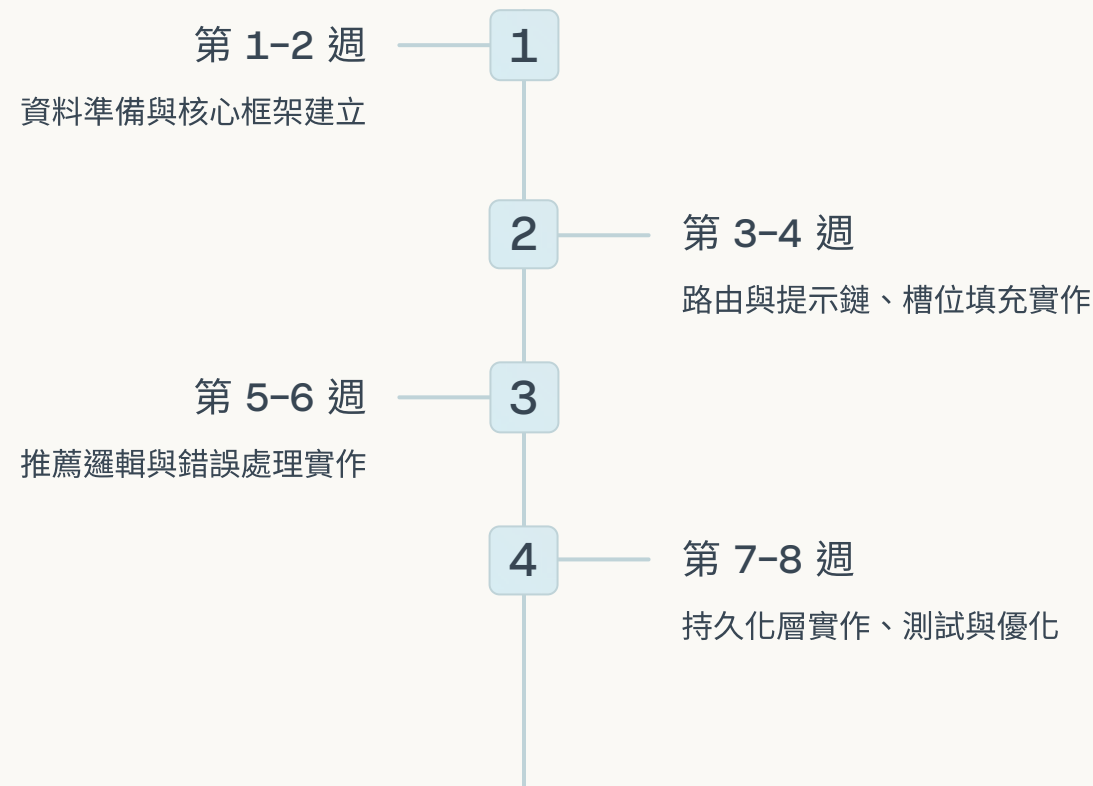
優化槽位結構

根據實際對話數據調整槽位定義，增加或移除槽位，優化必填標記。

提升檢索品質

優化向量檢索參數，提高推薦相關性，減少不相關或不適合的推薦。

實作時間表



這個時間表假設有專職團隊投入開發，可根據實際資源情況調整。關鍵里程碑包括基礎對話流程完成（第4週）和完整系統上線（第8週）。



總結與下一步

主要成果

- 完整的多輪引導式漏斗框架
- 智能化的中小型房車銷售助理
- 可擴展的對話系統架構

後續發展方向

- 整合更多銷售工具（預約試駕、報價單生成）
- 擴展至其他車型類別
- 增加多語言支援
- 實作 A/B 測試框架

這個框架不僅適用於中小型房車銷售，還可作為其他領域引導式銷售的基礎，具有廣泛的應用前景。